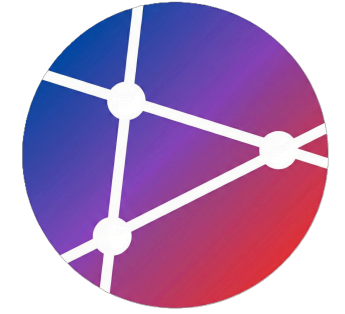




AI OPEN SOURCE SOFTWARE (OSS)

Automation with GitHub Actions

Week 6 Lecture Notes



Prof. Sejin Chun

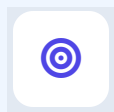
Dong-A University



SECTION 01

GitHub Actions로 자동화 시작하기

GitHub Actions의 기본 개념부터
본 강의의 전체적인 흐름을 안내합니다.



이 세션을 마치면...

GitHub Actions의 전체 아키텍처를 이해하고, 직접 CI 파이프라인을 구축하여 자동화된 개발 환경을 구성할 수 있게 됩니다.



01

아키텍처 이해

Workflow, Job, Step, Action 등 GitHub Actions의 핵심 구성 요소와 작동 원리 파악



02

YAML 작성

워크플로우 정의를 위한 YAML 문법을 익히고, 실제 실행 가능한 자동화 스크립트 작성



03

트리거 & 보안

Push, PR 등 다양한 이벤트 트리거 활용법과 Secrets를 통한 민감 정보 관리 방법 습득



04

CI 파이프라인

Node.js, Python 등 실제 프로젝트에 적용 가능한 기본 CI(지속적 통합) 파이프라인 구축

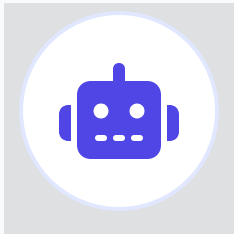


SECTION 02

GitHub Actions란?

정의부터 활용 배경까지
전체적인 개념 프레임을 소개합니다.

정의 및 용도



GitHub Actions

GitHub 저장소에서
직접 실행되는
자동화 플랫폼

개발 워크플로우를 저장소 내에서
이벤트 기반으로 자동 실행



CI/CD

지속적 통합 및 배포



테스트 자동화

단위/통합 테스트 실행



코드 품질 체크

Linting, Formatting



배포 자동화

AWS, Azure, Docker 등



이슈/PR 관리

라벨링, 자동 댓글



기타 자동화

모든 작업 자동화 가능



왜 GitHub Actions인가?

주요 장점

- ✓ GitHub 네이티브 통합
- ✓ Public 레포 무료
- ✓ 강력한 Marketplace 생태계
- ✓ 다양한 이벤트 트리거
- ✓ Matrix 빌드 지원
- ✓ 간단한 YAML 설정

기능	GitHub Actions	Jenkins	GitLab CI	CircleCI
설정 난이도	매우 쉬움	복잡	쉬움	쉬움
호스팅	GitHub 제공	자체 호스팅	GitLab 제공	클라우드
가격	무료 *	무료	무료 *	무료 *
VCS 통합	GitHub 전용	모든 VCS	GitLab 전용	모든 VCS
생태계	매우 큼	큼	중간	중간

* 일정 사용량까지 무료 제공

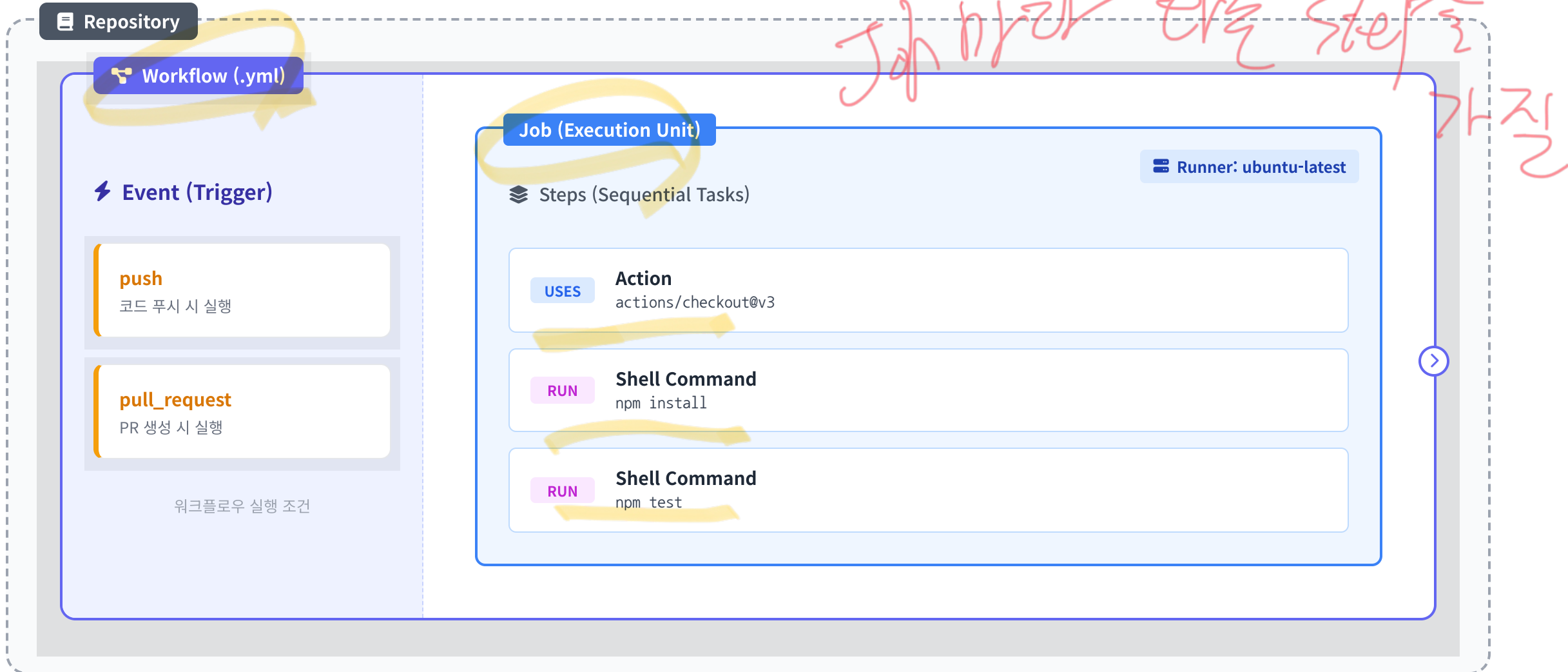


SECTION 03

GitHub Actions 아키텍처

Workflow, Job, Step, Runner 등
핵심 구성요소 간의 관계와 실행 구조를
상세히 알아봅니다.

아키텍처 핵심 개념



Workflow 예시 구조



.github/workflows/example.yml

```
name: My Workflow # 워크플로우 이름

on: [push, pull_request] # 트리거 이벤트

jobs: # 작업 정의
  build:
    runs-on: ubuntu-latest # 러너 환경
    steps: # 단계
      - uses: actions/checkout@v3
      - run: echo "Hello World"
```

name 워크플로우 이름

GitHub Actions 탭에 표시될 워크플로우의 식별 이름입니다. 명확하고 구분하기 쉬운 이름을 사용하는 것이 좋습니다.

on 이벤트 트리거

워크플로우가 언제 실행될지 정의합니다. `push`, `pull_request` 등 다양한 이벤트나 스케줄(cron)을 지정할 수 있습니다.

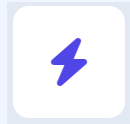
jobs 작업 정의

실제 실행될 작업들의 집합입니다. 각 작업(Job)은 기본적으로 병렬로 실행되며, `runs-on`으로 실행 환경(Runner)을 지정해야 합니다.

steps 실행 단계

Job 내에서 순차적으로 실행되는 명령들입니다. 미리 정의된 Action을 가져다 쓰거나(`uses`), 셸 명령어를 직접 실행(`run`)할 수 있습니다.

Event (이벤트)



워크플로우 트리거

워크플로우를 시작하는 **조건**입니다.

GitHub 저장소의 특정 활동이나 예약된 일정, 외부 이벤트 등이 방아쇠(Trigger)가 되어 자동화 작업을 실행합니다.



push

코드 커밋을 저장소에 푸시하거나 태그를 푸시할 때 실행됩니다. 가장 기본적인 CI 트리거입니다.



pull_request

Pull Request가 생성되거나 업데이트될 때 실행됩니다. 코드 리뷰 전 자동 테스트에 사용됩니다.



schedule

POSIX cron 구문을 사용하여 미리 정의된 시간에 주기적으로 워크플로우를 실행합니다.



workflow_dispatch

GitHub Actions 탭에서 버튼을 클릭하여 수동으로 워크플로우를 실행합니다. 입력값(inputs)을 받을 수 있습니다.



release

새로운 릴리스가 생성되거나 게시될 때 실행됩니다. 배포 자동화에 주로 사용됩니다.



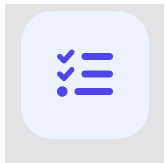
issues

이슈가 생성, 수정, 삭제되거나 라벨이 붙을 때 실행됩니다. 이슈 관리 자동화에 활용됩니다.

Job & Runner



Execution Unit

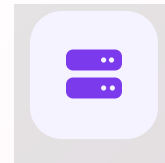


Job (작업)

워크플로우 내에서 실행되는 독립적인 작업 단위입니다. 기본적으로 병렬로 실행되어 속도를 최적화합니다.

- ✓ **병렬 실행 (기본값):** 여러 작업이 동시에 시작됨
- 🔗 **순차 실행 가능:** `needs` 키워드로 의존성 설정
- 🔧 **별도 러너 사용:** 각 Job은 깨끗한 가상 환경에서 실행
- 🔒 **의존성 정의:** 이전 작업 성공 시에만 실행되도록 제어

Environment



Runner (러너)

워크플로우의 각 Job을 실제로 실행하는 서버 환경입니다. GitHub이 제공하거나 직접 호스팅할 수 있습니다.

GitHub-hosted Runners

🖥️ ubuntu-latest, windows-latest, macos-latest 등 사전 설치된 환경 제공

Self-hosted Runners

🏠 자체 서버 사용, 특수한 하드웨어 설정이나 보안 네트워크 내부 실행 필요 시 사용



Step vs Action



EXECUTION UNIT

Step (단계)

Job 내에서 순차적으로 실행되는 개별 작업 단위입니다. 명령어를 직접 실행하거나 외부 액션을 가져와 사용합니다.

uses

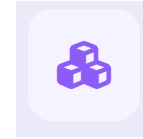
Action 사용

외부/내부 액션 로직 재사용

run

Shell 명령

Runner 환경에서 명령어 실행



REUSABLE UNIT

Action (액션)

복잡한 작업을 캡슐화하여 재사용 가능하게 만든 독립적인 작업 단위입니다. (Plugin 개념)

- ✓ **Public Actions** (Marketplace)
- ✓ **Private Actions** (자체 제작)
- ✓ Docker Container Actions
- ✓ JavaScript / Composite Actions



SECTION 05

첫 워크플로우 작성하기

Hello World 예제를 통해 워크플로우 파일의 위치,
기본 구조, 그리고 실행 과정을 단계별로 학습합니
다.



Hello World 워크플로우

.github/workflows/hello.yml

```
name: Hello World
```

```
# 트리거: push 이벤트 시
```

```
on: push
```

```
jobs:
```

```
  greet:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Say Hello
```

```
        run: echo "Hello, GitHub Actions!"
```

name 워크플로우 식별

Actions 탭에서 이 자동화 작업을 식별할 이름을 "Hello World"로 지정합니다. 가장 간단한 형태의 정의입니다.

on: push 실행 조건

저장소에 코드가 푸시(push)될 때마다 이 워크플로우가 자동으로 실행되도록 설정합니다.

runs-on 실행 환경

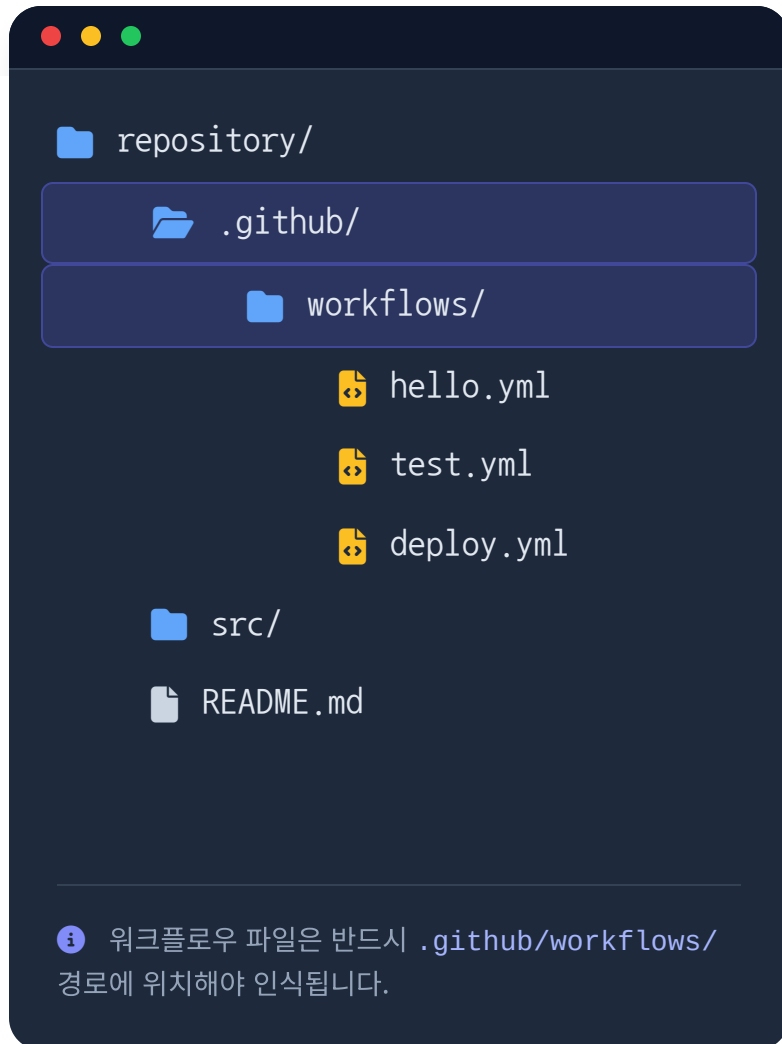
GitHub이 호스팅하는 최신 우분투(Ubuntu) 리눅스 환경에서 작업을 실행하도록 지정합니다.

run 명령 실행

터미널 셸 명령어를 실행합니다. 여기서는 "Hello, GitHub Actions!"라는 문자열을 로그에 출력합니다.



파일 위치 및 실행 과정



1 코드 푸시 (Push)

2 .github/workflows/ 확인

3 이벤트 매칭 워크플로우 검색

4 Runner (실행 서버) 할당

5 Job 실행 (병렬/순차)

6 Step별 명령/액션 수행

7 결과 리포트 및 로그 저장

자동화 완료 (Success/Fail)



SECTION 04

이벤트 트리거

주요 트리거 유형별 설정 패턴과
활용 방법을 알아봅니다.



Push 이벤트 패턴

Basic 모든 Push 이벤트

basic.yml

```
# 저장소의 모든 Push에 대해 실행
on: push
```

Branches 특정 브랜치 지정

branches.yml

```
on:
  push:
    branches:
      - 'main'
      - 'develop'
      - 'releases/**' # 와일드카드
```

Paths 파일 경로 필터링

paths.yml

```
on:
  push:
    paths:
      - 'src/**'
      - '**.js' # JS 파일 변경 시
      - '!docs/**' # 문서 제외
```

Ignore 제외 조건 (Ignore)

ignore.yml

```
on:
  push:
    branches-ignore:
      - 'feature/experimental'
    paths-ignore:
      - '**.md'
```

pull_request 트리거 설정



.github/workflows/pr-check.yml

1. 기본: PR 생성, 업데이트, 재오픈 시

on: pull_request

2. 활동 유형(types) 상세 제어

on:

pull_request:

types:

- opened # PR 생성 시
- synchronize # 새 커밋 푸시 시
- reopened # PR 다시 열림

3. 대상 브랜치 제한

on:

pull_request:

branches:

- main # main 브랜치 대상 PR만
- 'releases/**'

Default 기본 동작

별도 설정 없이 `on: pull_request` 만 선언하면, PR이 생성되거나(opened), 동기화되거나(synchronize), 다시 열릴 때(reopened) 자동으로 실행됩니다.

types 활동 유형 필터링

특정 활동에만 반응하도록 제어합니다. 예를 들어, 라벨이 붙을 때만 실행하거나 (`labeled`), 리뷰 요청이 있을 때만 실행하도록 (`review_requested`) 설정할 수 있습니다.

branches 대상 브랜치 제한

PR이 병합될 '도착지' 브랜치를 기준으로 필터링합니다. Feature 브랜치 간의 PR은 무시하고, `main` 이나 `develop` 등 중요 브랜치로 향하는 PR에만 검사를 수행할 때 유용합니다.

Schedule (Cron) 트리거



.github/workflows/schedule.yml

```
on:
  schedule:
    # 매일 자정 (UTC 기준)
    - cron: '0 0 * * *'

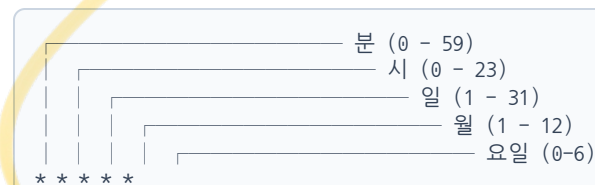
    # 매주 월요일 오전 9시 (UTC 기준)
    - cron: '0 9 * * 1'

    # 매월 1일 자정
    - cron: '0 0 1 * *'

    # 평일(월-금) 오전 5시 30분
    - cron: '30 5 * * 1-5'

jobs:
  daily-task:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Running scheduled task"
```

Cron 문법 구조



UTC 시간대 주의

GitHub Actions 스케줄은 **UTC(협정 세계시)** 기준입니다.
한국 시간(KST)은 UTC보다 9시간 빠릅니다.

예: 한국 오전 9시 = UTC 0시 ('0 0 * * *')

⚠ 제약 사항 및 팁

- **지연 실행:** 리소스 상황에 따라 다소 지연될 수 있습니다.
- **최소 간격:** 약 5분 간격이 최소 실행 주기입니다.
- **비활성화:** 60일간 활동 없으면 스케줄 자동 중지됩니다.



수동 실행 (workflow_dispatch)

```
.github/workflows/manual.yml

on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment to deploy'
        required: true
        default: 'staging'
        type: choice
        options:
          - staging
          - production

      log_level:
        description: 'Log level'
        required: false
        default: 'info'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo "Deploying to ${inputs.environment}"
          echo "Log level: ${inputs.log_level}"
```

workflow_dispatch 수동 트리거

GitHub 웹 인터페이스의 'Actions' 탭에서 버튼을 클릭하여 워크플로우를 수동으로 실행할 수 있게 합니다. 테스트나 비정기 배포 시 유용합니다.

inputs 입력 파라미터

실행 시 사용자로부터 입력받을 값을 정의합니다. `required` (필수 여부)와 `default` (기본값)를 설정할 수 있습니다.

type: choice 입력 타입

`type` 을 `choice` 로 설정하면 사용자가 직접 타이핑하는 대신 드롭다운 메뉴에서 옵션을 선택하도록 UI를 구성할 수 있습니다.

\${{ inputs.name }} 입력값 사용

입력받은 값은 `${{ inputs.변수명 }}` 문법을 통해 Job이나 Step 내에서 변수처럼 사용할 수 있습니다.

복합 이벤트 조건



.github/workflows/complex-events.yml

```
# 1. OR 조건 (여러 이벤트 중 하나)
# push 또는 pull_request 발생 시 실행
on: [push, pull_request]

# 2. AND 조건 (필터 조건 모두 만족)
# main 브랜치 AND src 폴더 변경 시에만 실행
on:
  push:
    branches:
      - main
  paths:
    - 'src/**'
```

OR 조건 다중 이벤트 트리거

여러 이벤트 중 **하나라도 발생하면** 워크플로우를 실행합니다. 대괄호 `[]` 를 사용하여 간단하게 이벤트 목록을 나열할 수 있습니다.

AND 조건 정밀한 필터링

하나의 이벤트 내에서 **모든 세부 조건이 만족될 때만** 실행됩니다. 위 예시는 'main' 브랜치에 푸시되면서 **동시에** 'src' 폴더 내의 파일이 변경된 경우에만 트리거됩니다.

💡 Best Practice

불필요한 빌드 실행을 방지하여 GitHub Actions 사용량(분)을 절약하려면 `paths` 나 `branches` 필터를 적극적으로 활용하세요.



SECTION 06

기본 Actions 활용

자주 사용되는 공식 Actions의
설정 방법과 활용 패턴을 알아봅니다.

actions/checkout@v3 활용



workflow.yml (Snippet)

steps:

1. 기본 코드 체크아웃 (필수!)

- uses: actions/checkout@v3

2. 전체 히스토리 가져오기

- uses: actions/checkout@v3

with:

fetch-depth: 0

3. 특정 브랜치 체크아웃

- uses: actions/checkout@v3

with:

ref: develop

Essential 기본 사용

GitHub Actions에서 가장 먼저 실행해야 하는 필수 단계입니다. Runner 환경(빈 서버)에 현재 저장소의 코드를 내려받아(Clone) 이후 단계에서 빌드나 테스트를 수행할 수 있게 합니다.

fetch-depth: 0 전체 히스토리

기본값은 1(최신 커밋만)입니다. 0으로 설정하면 전체 Git 히스토리 및 태그를 모두 가져옵니다. 버전 태그 기반 배포나 히스토리 분석 시 필요합니다.

ref 특정 브랜치/태그

워크플로우를 트리거한 브랜치가 아닌, 다른 특정 브랜치(develop), 태그, 또는 커밋 SHA를 명시적으로 체크아웃할 때 사용합니다.

Node.js 설정 및 캐싱



basic-setup.yml

```
steps:
# 1. 기본 버전 설정
- uses: actions/setup-node@v3
  with:
    node-version: '18'

# 3. 버전 범위 지정
- uses: actions/setup-node@v3
  with:
    node-version: '18.x'
# 최신 18.x 버전 자동 선택
```

caching.yml

```
steps:
# 2. 의존성 캐싱 (추천!)
- uses: actions/setup-node@v3
  with:
    node-version: '18'
    cache: 'npm'

# package-lock.json 기준
# 자동으로 ~/.npm 캐싱
# yarn, pnpm 지원
```

setup-node Node.js 환경 설정

공식 액션으로 Node.js를 설치하고 PATH에 추가합니다.
node-version 으로 버전을 명시하며, '18.x' 와 같이 메이저 버전만 지정하여 최신 패치를 유지하는 것이 유리합니다.

cache 의존성 캐싱

cache: 'npm' 옵션 하나로 node_modules 설치 시간을 획기적으로 줄일 수 있습니다. 별도의 actions/cache 설정 없이도 락파일 해시를 기반으로 자동 캐싱됩니다.

Python 환경 설정



```
.github/workflows/python-ci.yml

steps:
  - name: Setup Python
    uses: actions/setup-python@v4
    with:
      python-version: '3.10' # 버전 명시
      cache: 'pip' # pip 캐싱 활성화

  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt
```

uses actions/setup-python@v4

GitHub 공식 Python 설정 액션입니다. 특정 버전의 Python을 설치하고 PATH에 추가하여 후속 단계에서 `python` 명령어를 사용할 수 있게 합니다.

python-version 버전 지정

사용할 Python 버전을 지정합니다. `'3.10'` 과 같은 특정 버전이나 `'3.x'` 와 같은 시맨틱 버전을 지원하며, pypy도 설정 가능합니다.

cache 의존성 캐싱

`'pip'`, `'pipenv'`, `'poetry'` 등의 패키지 관리자를 지정하면 자동으로 의존성을 캐싱하여 빌드 속도를 획기적으로 개선합니다.

아티팩트 업/다운로드



Job 1: Build & Upload

Step: Upload

```
jobs:
  build:
    steps:
      - run: npm run build
        # 빌드 결과물 업로드
      - uses: actions/upload-artifact@v3
        with:
          name: build-output
          path: dist/
```

UPLOAD 결과물 저장

빌드 과정에서 생성된 dist/ 폴더를 GitHub 저장소의 아티팩트로 업로드합니다. 이 파일들은 워크플로우 실행이 끝난 후에도 다운로드하거나 다른 Job에서 사용할 수 있습니다.

Job 2: Deploy & Download

Step: Download

```
jobs:
  deploy:
    needs: build # build 완료 후 실행
    steps:
      # 빌드 결과물 다운로드
      - uses: actions/download-artifact@v3
        with:
          name: build-output
          path: dist/
```

DOWNLOAD 데이터 공유 및 가져오기

Job은 서로 다른 Runner(가상 머신)에서 실행되므로 파일 시스템이 공유되지 않습니다. download-artifact를 사용하여 이전 Job에서 업로드한 파일을 현재 Runner로 가져옵니다.



SECTION 07

Secrets 및 환경 변수

보안 정보 관리와 환경별 설정 구성을 위한
핵심 개념과 방법을 학습합니다.

Secrets 설정



설정 프로세스

-  **Step 1. 저장소 설정 이동**
GitHub 저장소 상단 탭에서 **Settings**를 클릭하여 이동합니다.
-  **Step 2. Secrets 메뉴 접근**
왼쪽 사이드바에서 **Secrets and variables** → **Actions**를 차례로 선택합니다.
-  **Step 3. 새 Secret 생성**
New repository secret 버튼을 클릭합니다.



주의사항

저장된 Secret 값은 다시 볼 수 없으므로, 생성 시 신중하게 입력하세요.

New secret

 **New repository secret**

Name

API_KEY

워크플로우에서 secrets.API_KEY로 참조할 이름입니다.

Secret

sk_live_51M3T...

Add secret

Secrets 사용 예시



steps-context.yml

```
steps:
  - name: Use secret
    run: |
      echo "API Key: ${ secrets.API_KEY }"
    env:
      API_KEY: ${ secrets.API_KEY }
```

`${ secrets.KEY }` Secrets 접근 컨텍스트

GitHub 저장소의 Settings > Secrets에 저장된 암호화된 값을 워크플로우 내에서 참조할 때 사용하는 문법입니다.

`env` 환경 변수 매핑

Secrets 값을 직접 쉘 명령에 하드코딩하지 않고, 환경 변수로 주입하여 스크립트 내에서 안전하게 사용할 수 있습니다.

Warning 보안 주의사항

로그 출력 시 Secrets 값은 자동으로 마스킹(***) 처리되지만, 가능한 한 값을 직접 출력하는 행위는 피해야 합니다.

환경 변수와 Context



.github/workflows/env-context.yml

1. 환경 변수 스코프 (Scope)

```
env:
  NODE_ENV: production # Workflow

jobs:
  build:
    env:
      DB_URL: postgres://localhost # Job
    runs-on: ubuntu-latest

    steps:
      - name: Build Step
        env:
          API_KEY: 123456 # Step (Highest)
        run: npm run build
```

2. GitHub 컨텍스트 활용

```
- name: Print Context
  run: |
    echo "Repo: ${{ github.repository }}"

    echo "Branch: ${{ github.ref }}"
    echo "Actor: ${{ github.actor }}"
    echo "SHA: ${{ github.sha }}"
    echo "Workflow: ${{ github.workflow }}"

    echo "Event: ${{ github.event_name }}"
```

ENV 변수 유효 범위 (Scope)

환경 변수는 **Workflow > Job > Step** 순으로 구체화되며, 하위 레벨이 상위 설정을 덮어씁니다(Override). 비밀 값은 반드시 Secrets를 사용하세요.

GITHUB.* 기본 제공 컨텍스트

워크플로우 실행 정보를 담고 있는 객체입니다. 동적인 조건 처리나 로깅에 필수적입니다.

repository: 소유자/저장소명
ref: 브랜치/태그 (refs/heads/main)
actor: 실행 트리거 사용자

</> \${{ expression }}

GitHub Actions 표현식 문법입니다. 컨텍스트 변수, 함수, 연산자 사용 시 중괄호 두 개로 감쌉니다.



SECTION 08

CI 워크플로우 실전 예시

Node.js, Python, Go 등 다양한 언어 환경에서의
실제 CI 파이프라인 구성 패턴을 알아봅니다.

Node.js 프로젝트 CI



.github/workflows/ci.yml

```
name: Node.js CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [16, 18, 20]

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-version }}
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test
```

on 이벤트 트리거

`push` (main, develop) 또는 `pull_request` (main) 발생 시 워크플로우가 자동으로 시작됩니다.

strategy Matrix 빌드

Node.js 16, 18, 20 버전에서 각각 테스트를 병렬로 수행합니다. `${{ matrix.node-version }}` 변수를 통해 각 버전을 동적으로 주입합니다.

uses 환경 설정 및 캐싱

`setup-node@v3` 액션을 사용하여 Node 환경을 구성하고, `cache: 'npm'` 옵션으로 의존성 설치 속도를 획기적으로 개선합니다.

run 의존성 설치 및 테스트

CI 환경에서는 `npm install` 대신 `npm ci` 를 사용하여 lock 파일 기반의 정확하고 빠른 설치를 보장합니다.

Python 프로젝트 CI



.github/workflows/python-ci.yml

```
name: Python CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ['3.9', '3.10', '3.11']

    steps:
      - uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: ${ matrix.python-version }
          cache: 'pip'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov pylint
```

strategy Matrix Build

여러 Python 버전(3.9, 3.10, 3.11)에서 병렬로 테스트를 실행합니다. 다양한 환경 호환성을 한 번에 검증할 수 있는 강력한 기능입니다.

setup-python 환경 설정 및 캐시

cache: 'pip' 옵션을 통해 의존성 설치 속도를 획기적으로 개선합니다. Matrix 변수를 활용해 각 버전에 맞는 Python을 설치합니다.

run Lint & Test

코드 품질 검사(Pylint)와 유닛 테스트(Pytest)를 순차적으로 실행합니다. 하나라도 실패하면 전체 Job이 실패 처리되어 문제를 즉시 알립니다.

codecov 커버리지 업로드

테스트 커버리지 결과를 Codecov와 같은 외부 서비스로 업로드하여, 코드 품질 추이를 시각적으로 모니터링할 수 있습니다.

Go 프로젝트 CI 워크플로우



.github/workflows/go-ci.yml

```
name: Go CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Go
        uses: actions/setup-go@v4
        with:
          go-version: '1.21'
          cache: true # 자동 캐싱 활성화

      - name: Install dependencies
        run: go mod download
```

setup-go Go 환경 설정

`cache: true` 옵션 하나로 Go 모듈 의존성을 자동으로 캐싱하여 빌드 속도를 획기적으로 개선합니다. 버전을 명시하여 일관된 환경을 보장합니다.

go mod 의존성 설치

`go mod download` 명령어를 사용하여 `go.mod` 와 `go.sum` 파일에 정의된 모든 의존성 패키지를 미리 다운로드합니다.

test -race 동시성 테스트

Go의 강력한 기능인 Race Detector(`-race`)를 활성화하여 동시성 관련 버그를 감지합니다. 커버리지 파일도 함께 생성합니다.

build 프로젝트 빌드

테스트가 모두 통과한 후, 전체 프로젝트를 빌드하여 컴파일 오류가 없는지 최종 확인합니다. `./...` 패턴으로 모든 패키지를 포함합니다.



SECTION 09

고급 워크플로우 패턴

Job 의존성 설정, 조건부 실행,
컨테이너 및 서비스 활용 등
심화 기능을 학습합니다.



Job 의존성 설정 (needs)

```
.github/workflows/dependencies.yml

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building..."

  test:
    needs: build # build 작업 완료 후 실행
    runs-on: ubuntu-latest
    steps:
      - run: echo "Testing..."

  deploy:
    needs: [build, test] # 둘 다 성공 시 실행
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying..."
```

needs 순차적 실행 제어

GitHub Actions의 Job은 기본적으로 병렬로 실행됩니다. **needs** 키워드를 사용하여 특정 Job이 완료된 후에만 실행되도록 순서를 지정할 수 있습니다.

Chain 단일 의존성

needs: build 와 같이 설정하면, **build** 작업이 완료된 후 **test** 작업이 시작됩니다. 이를 통해 빌드 아티팩트를 안전하게 전달합니다.

Array 다중 의존성 (Fan-in)

needs: [build, test] 처럼 배열 형태를 사용하여 여러 선행 작업이 모두 완료되어야 실행되도록 설정할 수 있습니다. 주로 배포(Deploy) 단계에서 사용됩니다.

조건부 실행 (Conditional Execution)



condition_example.yml

```
steps:
  - name: Run only on main
    if: github.ref == 'refs/heads/main'
    run: echo "Main branch"

  - name: Run only on PR
    if: github.event_name == 'pull_request'
    run: echo "Pull Request"

  - name: Run on success
    if: success() # 기본값 (생략 가능)
    run: echo "Previous steps succeeded"

  - name: Run on failure
    if: failure()
    run: echo "Something failed"

  - name: Always run
    if: always()
    run: echo "This always runs"
```

if 조건문 제어

Job이나 Step 레벨에서 `if` 키워드를 사용하여 실행 여부를 결정합니다. 조건이 `true` 일 때만 실행됩니다.

context 컨텍스트 활용

`github.ref` (브랜치), `github.event_name` (이벤트) 등 GitHub 컨텍스트 변수를 통해 실행 환경을 구체적으로 제어할 수 있습니다.

function 상태 확인 함수

- **success()**: 이전 단계 성공 시 (기본값)
- **failure()**: 이전 단계 실패 시 실행
- **always()**: 성공/실패 여부 무관하게 실행

💡 활용 사례

Main 브랜치 배포, 실패 시 슬랙 알림 전송, 테스트 실패 후에도 테스트 리포트 업로드 등에 유용하게 사용됩니다.

컨테이너 실행 환경 구성



.github/workflows/container.yml

```
jobs:
  container-job:
    runs-on: ubuntu-latest

    container: # 실행 환경 지정
      image: node:18 # Docker 이미지
      env:
        NODE_ENV: production

    steps:
      - uses: actions/checkout@v3
      - run: node --version
      - run: npm install
      - run: npm test
```

container 컨테이너 환경 정의

Job을 실행할 Docker 컨테이너를 지정합니다. 호스트 머신(Runner) 위에서 지정된 컨테이너가 실행되며, 모든 Step은 이 컨테이너 내부에서 수행됩니다.

image Docker 이미지

Docker Hub 또는 Private Registry의 이미지를 사용할 수 있습니다. 예시에서는 `node:18` 이미지를 사용하여 별도의 Node.js 설치 과정 없이 바로 환경을 구축합니다.

✓ 주요 장점

프로젝트에 필요한 특정 도구나 버전을 일관성 있게 관리할 수 있습니다. CI 실행 환경과 로컬/프로덕션 환경을 Docker 이미지로 통일하여 "내 컴퓨터에선 되는데..." 문제를 방지합니다.



서비스 컨테이너 (Service Containers)

service-containers.yml

```
jobs:
  test:
    runs-on: ubuntu-latest

    services: # 서비스 컨테이너 정의
      postgres:
        image: postgres:14
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

      redis:
        image: redis:7
        options: >-
          --health-cmd "redis-cli ping"
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v3
      - run: npm test
      env:
        DATABASE_URL: postgres://
```

services 서비스 컨테이너 정의

테스트나 빌드에 필요한 데이터베이스(PostgreSQL), 캐시(Redis) 등을 Docker 컨테이너로 띄웁니다. 워크플로우 실행 환경에 쉽게 통합됩니다.

options 헬스 체크 (Health Check)

컨테이너가 완전히 준비될 때까지 기다리기 위해 `--health-cmd` 옵션을 사용합니다. DB 초기화 지연으로 인한 테스트 실패를 방지합니다.

ports 포트 매핑

서비스 컨테이너의 포트를 호스트(Runner) 포트와 연결합니다. `5432:5432` 와 같이 설정하여 로컬호스트처럼 접근할 수 있습니다.

localhost 네트워크 접근

서비스 컨테이너는 Runner와 동일한 네트워크를 공유합니다. 따라서 애플리케이션 설정에서 `localhost` 를 사용하여 DB나 Redis에 접속합니다.



SECTION 10

실습 과제

과제를 통해 실제 CI/CD 워크플로우를 구성하고
자동화 파이프라인을 직접 구축해봅니다.



과제 1: 기본 CI 구축

MISSION 01

Hello, CI Pipeline!

가장 기본적인 형태의 CI 워크플로우를 직접 작성해보며 자동화의 기초를 다집니다. 코드가 푸시될 때마다 자동으로 실행되는 파이프라인을 경험해보세요.

🕒 20 Mins 📶 Easy 👤 Individual



1

프로젝트 생성

간단한 Node.js 또는 Python 프로젝트를 생성합니다. `npm init` 또는 `requirements.txt` 를 포함하여 의존성 관리 환경을 만드세요.



2

워크플로우 파일 작성

저장소 루트에 `.github/workflows/ci.yml` 파일을 생성하고, `on: push` 이벤트를 트리거로 설정합니다.



3

자동화 단계 구성

Lint(코드 검사)와 Test(단위 테스트)를 실행하는 Job을 정의합니다. `actions/checkout` 과 언어별 setup 액션을 사용하세요.



4

실행 및 검증

코드를 GitHub에 Push한 후, 저장소의 **Actions** 탭에서 워크플로우가 자동으로 실행되고 성공(녹색 체크)하는지 확인합니다.



과제 2: Matrix 빌드

MISSION 02

Test Across Environments

하나의 설정으로 여러 환경을 동시에 테스트합니다. Matrix 전략을 사용하여 다양한 Node.js 버전 및 운영체제 호환성을 검증하는 파이프라인을 구축하세요.

🕒 25 Mins 📊 Medium 👤 Individual



1

Node 버전 매트릭스

테스트할 Node.js 버전을 배열로 정의합니다.
`node-version: [16, 18, 20]` 을 사용하여 여러 버전에서 동시에 테스트가 실행되도록 설정합니다.



2

OS 매트릭스 확장

실행 환경(Runner)을 Ubuntu, Windows, macOS 등으로 확장합니다. `os: [ubuntu-latest, windows-latest]` 를 추가하여 크로스 플랫폼을 검증하세요.



3

전략(Strategy) 구성

Job 레벨에서 `strategy` 키워드를 사용하여 매트릭스 변수를 정의하고, steps에서 `${{matrix.node-version}}` 형태로 참조합니다.



4

결과 비교 분석

Actions 탭에서 조합(N x M)만큼 생성된 하위 작업들이 병렬로 실행되는 것을 확인하고, 특정 환경에서의 성공 여부를 비교 분석합니다.

과제 3: Secrets 활용



MISSION 03

Secure Your Secrets

API 키나 비밀번호 같은 민감한 정보를 안전하게 관리하는 방법을 익힙니다. 코드로 노출하지 않고 환경 변수로 주입하는 실습입니다.

🕒 15 Mins 📊 Medium 👤 Individual



1

GitHub Secrets 설정

저장소의 **Settings > Secrets and variables > Actions** 메뉴로 이동하여 `API_KEY` 와 같은 이름으로 새 Secret을 등록합니다.



2

워크플로우에서 호출

YAML 파일 내에서 `${{ secrets.API_KEY }}` 문법을 사용하여 등록된 Secret 값을 불러오도록 작성합니다.



3

환경 변수 매핑

`env` 키워드를 사용하여 Secret 값을 환경 변수로 매핑하거나, Step의 입력값(`with`)으로 전달합니다.



4

보안 확인

실행 로그에서 Secret 값이 직접 노출되지 않고 `***` 로 마스킹되어 안전하게 처리되었는지 확인합니다.



과제 4: 복합 워크플로우

MISSION 04

Complex Workflow

실제 배포 환경과 유사한 파이프라인을 구축합니다. Job 간의 의존성을 설정하여 순차적으로 실행되게 하고, 조건에 따라 배포가 결정되는 로직을 구현해봅니다.

🕒 40 Mins 📶 Hard 👤 Individual



1

3단계 파이프라인 구성

워크플로우 내에 `build`, `test`, `deploy` 라는 3개의 별도 Job을 정의합니다.



2

Job 의존성 설정

`needs` 키워드를 사용하여 빌드가 성공해야 테스트가 돌고, 테스트가 성공해야 배포가 실행되도록 연결합니다.



3

조건부 배포 설정

Deploy Job에 `if` 조건을 추가하여, `main` 브랜치일 때만 배포가 실행되도록 제한합니다.



4

아티팩트 전달

빌드 결과물을 `upload-artifact` 로 저장하고, 배포 단계에서 `download-artifact` 로 가져와 사용합니다.



SECTION 11

참고 자료 References

공식 문서와 액션 마켓플레이스 탐색하여
지식을 확장하고 심화 학습을 진행합니다.



GitHub Actions Docs

GitHub Actions의 모든 기능, 개념, 가이드가 포함된 공식 문서입니다. 가장 기본이 되는 학습 자료입니다.

docs.github.com/actions



Workflow Syntax

YAML 파일 작성을 위한 구문 레퍼런스입니다. `on`, `jobs`, `steps` 등 키워드별 상세 명세를 제공합니다.

docs.github.com/.../workflow-syntax...



Actions Marketplace

커뮤니티에서 제작한 수천 개의 재사용 가능한 Action을 검색하고 적용할 수 있는 마켓플레이스입니다.

github.com/marketplace?type=actions

유용한 Actions & 리소스



Essential Actions



actions/checkout Essential

저장소의 코드를 Runner 환경으로 내려받는 필수 액션



actions/setup-node

Node.js 환경 설정 및 npm 패키지 캐싱 지원



actions/cache

의존성 파일을 캐싱하여 빌드 속도를 획기적으로 단축



codecov/codecov-action

테스트 커버리지 리포트를 Codecov 서비스로 자동 업로드



Learning Resources



GitHub Skills

GitHub에서 제공하는 인터랙티브한 실습형 학습 코스
skills.github.com



Awesome Actions

커뮤니티가 선정한 유용한 GitHub Actions 모음집
github.com/sdras/awesome-actions



Actions Marketplace

수천 개의 커뮤니티 액션을 검색하고 적용할 수 있는 마켓



Workflow Syntax

YAML 파일 작성에 필요한 공식 문법 레퍼런스 가이드





SECTION 12

핵심 요약

Key Takeaways

GitHub Actions의 핵심 개념 정리 및
효율적인 자동화를 위한 Best Practices

핵심 요약



Workflow Structure

Event(트리거) + Jobs(작업) + Steps(단계)의 계층 구조를 명확히 이해해야 합니다.



Checkout First

대부분의 작업에서 actions/checkout@v3는 필수적인 첫 단계입니다.



Security First

API 키나 토큰은 절대 코드에 하드코딩하지 말고 **Secrets**를 사용하세요.



Matrix Builds

다양한 OS와 언어 버전에서 동시에 테스트하여 호환성을 확보하세요.

Best Practices

- ✓ 워크플로우는 작고 집중적으로 유지
- ✓ 재사용 가능한 Actions 적극 활용
- ✓ Fail Fast 전략으로 리소스 절약
- ✓ 의존성 캐싱으로 빌드 속도 향상
- ✓ 명확한 Job/Step 네이밍 사용

“

*"Automate everything you can."
- DevOps Principle*



다음 단계 및 예고



CI 워크플로우 구축

실제 진행 중인 프로젝트에 CI/CD 파이프라인을 직접 적용해 보세요.



Marketplace 탐색

GitHub Marketplace에서 유용하고 검증된 Actions를 찾아 활용하세요.



커스텀 Action 제작

복잡한 스크립트를 재사용 가능한 나만의 Action으로 모듈화해 보세요.



워크플로우 최적화

불필요한 단계를 줄이고 캐싱을 활용하여 실행 시간을 단축하세요.



Next Week Preview



다음 주차 강의 상세 내용은
LMS 수업 공지사항을 참조해 주세요.



금주 실습 과제 제출 기한 확인



Q&A 및 프로젝트 멘토링 준비



"Automate everything you can."

- DevOps Principle

자동화로 더 중요한 일에 집중하세요! 🚀